

## 1. Create Numeric, Character and Logical Vectors

### Aim

To create numeric, character and logical vectors in R and display their type and contents.

### Procedure

1. Create a numeric vector using `c()`.
2. Create a character vector using `c()`.
3. Create a logical vector using `TRUE` and `FALSE`.
4. Display the contents and data type using `print()` and `typeof()`.

### Code

```
# Numeric vectornum <- c(10, 20, 30, 40, 50)
# Character vectorchar <- c("Apple", "Banana", "Mango", "Orange")
# Logical vectorlogical <- c(TRUE, FALSE, TRUE, FALSE)
# Display contents and typeprint(num)print(typeof(num))
print(char)print(typeof(char))
print(logical)print(typeof(logical))
```

### Result

Thus, numeric, character and logical vectors were created successfully and their contents and types were displayed.

---

## 2. Create Labeled Matrices

### Aim

To create labeled matrices of sizes  $5 \times 4$ ,  $3 \times 3$  and  $2 \times 2$  using row-wise and column-wise filling.

### Procedure

1. Create matrices using the `matrix()` function.
2. Use `byrow=TRUE` for row-wise filling.
3. Use `byrow=FALSE` for column-wise filling.
4. Assign row and column names.
5. Display the matrices.

### Code

```
# 5 x 4 matrix - filled by rowm1 <- matrix(1:20, nrow=5, ncol=4, byrow=TRUE)rownames(m1) <- paste0("R",
1:5)colnames(m1) <- paste0("C", 1:4)
# 3 x 3 matrix - filled by columnm2 <- matrix(1:9, nrow=3, ncol=3, byrow=FALSE)rownames(m2) <- paste0("R",
1:3)colnames(m2) <- paste0("C", 1:3)
# 2 x 2 matrix - filled by rowm3 <- matrix(1:4, nrow=2, ncol=2, byrow=TRUE)rownames(m3) <- c("A",
"B")colnames(m3) <- c("X", "Y")
print(m1)print(m2)print(m3)
```

### Result

Thus, labeled  $5 \times 4$ ,  $3 \times 3$  and  $2 \times 2$  matrices were created and displayed successfully.

---

### 3. Create and Display a 3D Array

#### Aim

To create and display a three-dimensional array with specified rows, columns and tables.

#### Procedure

1. Create an array using the array() function.
2. Specify dimensions using dim.
3. Store values from 1 to 12.
4. Display the array and its dimensions.

#### Code

```
# Create 3D arrayarr <- array(1:12, dim=c(2,3,2))
# Display arrayprint(arr)
# Display dimensionsprint(dim(arr))
```

#### Result

Thus, a 3D array containing 2 rows, 3 columns and 2 tables was created and displayed successfully.

---

### 4. Create Arrays from Vectors with Dimension Names

#### Aim

To create an array from a vector with dimension names and access specific elements.

#### Procedure

1. Create a vector containing values.
2. Convert it into a 3D array.
3. Assign row, column and table names.
4. Access specific elements using indices.

#### Code

```
# Create vectorv <- 1:12
# Create named 3D arrayarr <- array(
  v,
  dim=c(2,3,2),
  dimnames=list(
    c("Row1","Row2"),
    c("Col1","Col2","Col3"),
    c("Table1","Table2")
  )
)
# Display arrayprint(arr)
# Print specific elementsprint(arr[1,2,1])print(arr[2,3,2])
```

#### Result

Thus, an array was created from a vector with dimension names and specific elements were accessed successfully.

---

## 5. Create and Manipulate Factor Variables

### Aim

To create and manipulate factor variables and generate a random sample of letters in R.

### Procedure

1. Create a vector containing height values.
2. Create a categorical gender factor.
3. Generate random letters using sample().
4. Convert the letters into a factor.
5. Display the factor levels.

### Code

```
# Numeric vector heights <- c(160, 165, 170, 155, 168)
# Factor variable gender <- factor(c("Female", "Male", "Female", "Female", "Male"))
# Random LETTERS sample letters_sample <- sample(LETTERS, 10)
# Convert letters to factor letter_factor <- factor(letters_sample)
print(heights) print(gender) print(letters_sample) print(letter_factor) print(levels(letter_factor))
```

### Result

Thus, factor variables were created and manipulated successfully, and random letters were generated.

---

## 6. Create an R List

### Aim

To create an R list containing vectors, matrices and functions and display its contents.

### Procedure

1. Create a vector.
2. Create a matrix.
3. Define a function.
4. Store all three objects in a list.
5. Access and display individual list elements.

### Code

```
# Create vector v <- c(10,20,30,40)
# Create matrix m <- matrix(1:9, nrow=3)
# Create function square <- function(x) {
  return(x*x)
}
# Create list mylist <- list(
  Vector=v,
  Matrix=m,
  Function=square
)
# Display list print(mylist)
# Access list elements print(mylist$Vector) print(mylist$Matrix) print(mylist$Function(5))
```

### Result

Thus, an R list containing a vector, matrix and function was created and its contents were displayed successfully.

---

## 7. Factors, Random Integers and FizzBuzz

### Aim

To find the factors of a number, generate random integers and implement the FizzBuzz logic using R.

### Procedure

1. Use a loop to find factors of a number.
2. Generate 10 random integers between -50 and 50.
3. Use a loop from 1 to 100.
4. Apply FizzBuzz conditions using the modulus operator.

### Code

```
# Find factors
n <- 24
factors <- c()
for(i in 1:n) {
  if(n %% i == 0) {
    factors <- c(factors, i)
  }
}
print(factors)

# Generate 10 random integers
random_numbers <- sample(-50:50, 10)
print(random_numbers)

# FizzBuzz
for(i in 1:100) {
  if(i %% 15 == 0) {
    print("FizzBuzz")
  } else if(i %% 3 == 0) {
    print("Fizz")
  } else if(i %% 5 == 0) {
    print("Buzz")
  } else {
    print(i)
  }
}
```

### Result

Thus, the factors were found, random integers were generated, and FizzBuzz logic was implemented successfully.

---

## 8. Generate Random Numbers from Normal Distribution

### Aim

To generate random numbers from a normal distribution and count their occurrences based on specified ranges.

### Procedure

1. Generate 100 random numbers using `rnorm()`.
2. Set the mean and standard deviation.
3. Count values greater than 50 and less than 50.
4. Group the values into intervals using `cut()` and count them.

### Code

```

set.seed(123)
# Generate normal random numbers x <- rnorm(100, mean=50, sd=10)
# Display numbers print(x)
# Count occurrences greater_50 <- sum(x > 50) less_50 <- sum(x < 50)
print(greater_50) print(less_50)
# Count values in intervals counts <- table(
  cut(x, breaks=c(-Inf,40,50,60,Inf))
)
print(counts)

```

### Result

Thus, normally distributed random numbers were generated and their occurrences in different ranges were counted successfully.

---

## 9. Create Empty Plot with Axis Limits

### Aim

To create an empty plot with specified X-axis and Y-axis limits.

### Procedure

1. Use the plot() function.
2. Set type="n" to create an empty plotting region.
3. Specify X-axis and Y-axis limits using xlim and ylim.
4. Add axis labels and title.

### Code

```

plot(
  0, 0,
  type="n",
  xlim=c(0,100),
  ylim=c(0,100),
  xlab="X Axis",
  ylab="Y Axis",
  main="Empty Plot"
)

```

### Result

Thus, an empty plot with the specified axis limits and labels was created successfully.

---

## 10. Create and Explore exam\_data

### Aim

To create and manipulate a data frame named exam\_data by extracting data, adding rows and columns, sorting and saving it to a file.

### Procedure

1. Create a data frame with name, score, attempts and qualify.
2. Display its structure.

3. Extract required rows and columns.
4. Add a new row and column.
5. Sort the data.
6. Save the data frame as a CSV file.

#### Code

```
# Create data frame
exam_data <- data.frame(
  name=c("Arun","Bala","Chitra","Divya","Elan"),
  score=c(85,72,91,65,78),
  attempts=c(1,2,1,3,2),
  qualify=c(TRUE,TRUE,TRUE,FALSE,TRUE)
)
# Display data
print(exam_data)
# Structure
str(exam_data)
# Extract score
print(exam_data$score)
# Extract first row
print(exam_data[1,])
# Add new row
new_row <- data.frame(
  name="Fathima",
  score=88,
  attempts=1,
  qualify=TRUE
)
exam_data <- rbind(exam_data, new_row)
# Add new column
exam_data$grade <- ifelse(
  exam_data$score >= 80, "A", "B"
)
# Sort by score
exam_data <- exam_data[order(-exam_data$score),]
print(exam_data)
# Save to CSV
write.csv(exam_data, "exam_data.csv", row.names=FALSE)
```

#### Result

Thus, the exam\_data data frame was created, manipulated, sorted and successfully saved as a CSV file.

## 11. Read a CSV File and Display Contents

#### Aim

To read a CSV file in R and display its contents.

#### Procedure

1. Use read.csv() to read the CSV file.
2. Store the data in a data frame.
3. Display the data.
4. Display its structure and summary.

#### Code

```
# Select CSV file manually
data <- read.csv(file.choose())
# Display contents
print(data)
# Display structure
str(data)
# Display summary
summary(data)
```

#### Result

Thus, the CSV file was successfully read and its contents, structure and summary were displayed.

**Note:** When running this program, select your .csv file from the file-selection window.

---

## 12. Data Reshaping Using Melt and Cast

### Aim

To reshape an air-quality dataset using melt and cast operations and calculate monthly averages of Ozone, Solar Radiation, Wind and Temperature.

### Procedure

1. Load the airquality dataset.
2. Load the reshape2 package.
3. Convert the data into long format using melt().
4. Convert it into summarized wide format using dcast().
5. Calculate monthly averages.

### Code

```
# Install once if required:# install.packages("reshape2")
library(reshape2)
# Load airquality datasetdata <- airquality
# Melt datamelted <- melt(
  data,
  id.vars=c("Month","Day"),
  measure.vars=c("Ozone","Solar.R","Wind","Temp")
)
print(melted)
# Cast data and calculate monthly averagescasted <- dcast(
  melted,
  Month ~ variable,
  fun.aggregate=mean,
  na.rm=TRUE
)
print(casted)
# Monthly averagesmonthly_avg <- aggregate(
  cbind(Ozone, Solar.R, Wind, Temp) ~ Month,
  data=data,
  FUN=mean,
  na.rm=TRUE
)
print(monthly_avg)
```

### Result

Thus, the air-quality dataset was reshaped using melt and cast operations and monthly averages were calculated successfully.

---

## 13. Combine Multiple Arrays Row-Wise

### Aim

To combine multiple arrays or matrices row-wise using the rbind() function.

### Procedure

1. Create three matrices.
2. Store different values in each matrix.
3. Combine the matrices using rbind().
4. Display the resulting matrix.

#### Code

```
# Create matricesa1 <- matrix(1:6, nrow=2, ncol=3)a2 <- matrix(7:12, nrow=2, ncol=3)a3 <- matrix(13:18,
nrow=2, ncol=3)
# Display matricesprint(a1)print(a2)print(a3)
# Combine row-wiseresult <- rbind(a1, a2, a3)
print(result)
```

#### Result

Thus, multiple matrices were successfully combined row-wise into a single matrix.

---

### 14. Explore and Manipulate ChickWeight

#### Aim

To explore and manipulate the ChickWeight dataset using sorting, melting and casting operations.

#### Procedure

1. Load the built-in ChickWeight dataset.
2. Display its structure and summary.
3. Sort the data according to weight and diet.
4. Melt the dataset.
5. Cast the melted data to calculate average weight.

#### Code

```
# Load datasetdata <- ChickWeight
# Display datasetprint(data)
# Structure and summarystr(data)summary(data)
# Sort by weightsorted_weight <- data[order(data$weight),]print(sorted_weight)
# Sort by Dietsorted_diet <- data[order(data$Diet),]print(sorted_diet)
# Melt datalibrary(reshape2)
melted <- melt(
  data,
  id.vars=c("Chick","Time","Diet")
)
print(melted)
# Cast datacasted <- dcast(
  melted,
  Diet ~ Time,
  value.var="value",
  fun.aggregate=mean
)
print(casted)
```

#### Result

Thus, the ChickWeight dataset was explored, sorted, melted and cast successfully.

---

## 15. EDA on Iris Dataset

### Aim

To perform exploratory data analysis on the Iris dataset using dimensions, summary, standard deviation, quantiles, grouping and boxplots.

### Procedure

1. Load the Iris dataset.
2. Find its dimensions.
3. Generate summary statistics.
4. Calculate standard deviation and quantiles.
5. Group the data according to species.
6. Visualize the data using a boxplot.

### Code

```
# Load Iris datasetdata <- iris
# Dimensionsprint(dim(data))
# Summaryprint(summary(data))
# Standard deviationprint(sapply(data[1:4], sd))
# Quantilesprint(sapply(data[1:4], quantile))
# Mean Sepal Length by Speciesgrouped <- aggregate(
  Sepal.Length ~ Species,
  data=data,
  FUN=mean
)
print(grouped)
# Pivot tableprint(table(data$Species))
# Boxplotboxplot(
  Sepal.Length ~ Species,
  data=data,
  main="Sepal Length by Species",
  xlab="Species",
  ylab="Sepal Length"
)
```

### Result

Thus, exploratory data analysis was successfully performed on the Iris dataset using statistical measures, grouping, tables and visualization.

---

## 16. Explore USArrests Dataset

### Aim

To analyze the USArrests dataset using summary statistics, maximum and minimum values, correlation, filtering and graphical visualization.

### Procedure

1. Load the USArrests dataset.
2. Calculate summary statistics.
3. Find the state with the largest rape arrest rate.
4. Find maximum and minimum murder rates.
5. Calculate correlation between features.

6. Identify states above median assault arrests and bottom 25% murder arrests.
7. Generate graphs.

### Code

```
# Load datasetdata <- USArrests
# Summary statisticsprint(summary(data))
# Meanprint(colMeans(data))
# Standard deviationprint(sapply(data, sd))
# State with largest rape arrestsstate_rape <- rownames(data)[which.max(data$Rape)]
print(state_rape)print(max(data$Rape))
# Maximum and minimum murder ratesprint(max(data$Murder))print(min(data$Murder))
# Correlationprint(cor(data))
# States above median Assaultmedian_assault <- median(data$Assault)
above_median <- data[
  data$Assault > median_assault,
]
print(above_median)
# Bottom 25% for Murderq25 <- quantile(data$Murder, 0.25)
bottom25 <- data[
  data$Murder <= q25,
]
print(bottom25)
# Histogramhist(
  data$Murder,
  main="Murder Arrests",
  xlab="Murder"
)
# Density plotplot(
  density(data$Murder),
  main="Density of Murder Arrests"
)
# Scatter plotplot(
  data$Murder,
  data$Assault,
  main="Murder vs Assault",
  xlab="Murder",
  ylab="Assault"
)
# Bar graphbarplot(
  data$Murder[1:10],
  names.arg=rownames(data)[1:10],
  las=2,
  main="Murder Arrests"
)
```

### Result

Thus, the USArrests dataset was successfully analyzed using statistical calculations, filtering, correlation and different visualization techniques.

---

## 17. Explore Titanic Dataset

### Aim

To analyze the Titanic dataset using bar charts for survival and gender and a histogram for passenger age.

### Procedure

1. Load the Titanic dataset.

2. Convert the contingency table into a data frame.
3. Calculate survival frequencies.
4. Create a bar chart for survival.
5. Analyze survival by gender.
6. Use an age-containing Titanic dataset to create an age histogram.

### Code

```
# Titanic survival dataset
data <- as.data.frame(Titanic)
print(data)
# Survival counts
survival <- aggregate(
  Freq ~ Survived,
  data=data,
  sum
)
print(survival)
# Bar chart of survival
barplot(
  survival$Freq,
  names.arg=survival$Survived,
  main="Titanic Survival",
  xlab="Survived",
  ylab="Frequency"
)
# Survival by gender
gender_survival <- tapply(
  data$Freq,
  list(data$Sex, data$Survived),
  sum
)
print(gender_survival)
barplot(
  gender_survival,
  beside=TRUE,
  legend=TRUE,
  main="Survival by Gender",
  xlab="Gender",
  ylab="Frequency"
)
# Titanic dataset with individual passenger ages
if(!require(titanic, quietly=TRUE)) {
  install.packages("titanic")
  library(titanic)
} else {
  library(titanic)
}
age_data <- titanic_train
# Histogram of Age
hist(
  age_data$Age,
  main="Histogram of Passenger Age",
  xlab="Age",
  ylab="Frequency"
)
```

### Result

Thus, the Titanic dataset was explored using survival and gender bar charts and a histogram of passenger ages.

**Note:** The built-in R Titanic dataset does not contain individual passenger ages. Therefore, the titanic package is used only for the Age histogram.

---

## 18. Create Basic Graphs in R

### Aim

To create boxplot, histogram, bar plot, line chart and scatter plot using R.

### Procedure

1. Use the Iris dataset.
2. Create a boxplot for Sepal Length.
3. Create a histogram.
4. Create a bar plot for species.
5. Create a line chart.
6. Create a scatter plot between Sepal Length and Petal Length.

### Code

```
# Load Iris dataset
data <- iris

# Boxplot
boxplot(
  data$Sepal.Length,
  main="Boxplot of Sepal Length",
  ylab="Sepal Length"
)

# Histogram
hist(
  data$Sepal.Length,
  main="Histogram of Sepal Length",
  xlab="Sepal Length",
  ylab="Frequency"
)

# Bar plot
barplot(
  table(data$Species),
  main="Species Count",
  xlab="Species",
  ylab="Count"
)

# Line chart
plot(
  data$Sepal.Length,
  type="l",
  main="Line Chart of Sepal Length",
  xlab="Observation",
  ylab="Sepal Length"
)

# Scatter plot
plot(
  data$Sepal.Length,
  data$Petal.Length,
  main="Sepal Length vs Petal Length",
  xlab="Sepal Length",
  ylab="Petal Length"
)
```

### Result

Thus, boxplot, histogram, bar plot, line chart and scatter plot were successfully created in R.

---

## 19. Regression Model on Advertising Dataset

### Aim

To build a simple linear regression model using advertising spend to predict sales.

### Procedure

1. Create an advertising dataset containing Spend and Sales.

2. Build a linear regression model using `lm()`.
3. Display the model summary.
4. Predict Sales for a new advertising spend value.
5. Plot the regression line.

#### Code

```
# Create Advertising dataset
advertising <- data.frame(
  Spend=c(100,150,200,250,300,350,400,450,500,550),
  Sales=c(20,28,35,42,50,55,63,70,78,85)
)
# Build regression model
model <- lm(
  Sales ~ Spend,
  data=advertising
)
# Display model summary
print(summary(model))
# Predict Sales for Spend = 600
new_data <- data.frame(Spend=600)
prediction <- predict(
  model,
  newdata=new_data
)
print(prediction)
# Scatter plot
plot(
  advertising$Spend,
  advertising$Sales,
  main="Advertising Spend vs Sales",
  xlab="Advertising Spend",
  ylab="Sales"
)
# Regression line
abline(model)
```

#### Result

Thus, a simple linear regression model was built successfully and Sales was predicted based on advertising Spend.

## 20. Multiple Regression Using ChickWeight

#### Aim

To build a multiple regression model using Time and Diet as predictors to predict the weight of chicks.

#### Procedure

1. Load the ChickWeight dataset.
2. Use Weight as the dependent variable.
3. Use Time and Diet as independent variables.
4. Build a multiple linear regression model using `lm()`.
5. Predict the weight for a new observation.
6. Compare actual and predicted values graphically.

#### Code

```
# Load ChickWeight dataset
data <- ChickWeight
# Build multiple regression model
model <- lm(
  weight ~ Time + Diet,
  data=data
)
# Display model summary
print(summary(model))
```

```
# New observationnew_data <- data.frame(
  Time=20,
  Diet=factor("1", levels=levels(data$Diet))
)
# Predict weightprediction <- predict(
  model,
  newdata=new_data
)
print(prediction)
# Predict all observationspredicted <- predict(model)
# Actual vs predicted plotplot(
  data$weight,
  predicted,
  main="Actual vs Predicted Weight",
  xlab="Actual Weight",
  ylab="Predicted Weight"
)
abline(0,1)
```

## Result

Thus, a multiple regression model was created successfully using Time and Diet, and chick weight was predicted.

## 21. Iris Train/Test Split and Logistic Regression

### Aim

To randomly divide the Iris dataset into 80% training and 20% testing data, build a logistic regression model, predict the class and evaluate it using a confusion matrix.

### Procedure

1. Load the Iris dataset.
2. Convert the three-class Species variable into a binary classification problem: Setosa and Non-Setosa.
3. Randomly split the dataset into 80% training and 20% testing data.
4. Build a logistic regression model using Petal Length and Petal Width.
5. Predict the test data.
6. Create a confusion matrix and calculate accuracy.

### Code

```
# Load Iris datasetdata <- iris
# Convert to binary classificationdata$Species <- ifelse(
  data$Species == "setosa",
  "setosa",
  "non-setosa"
)
data$Species <- factor(data$Species)
# Random 80/20 splitset.seed(123)
n <- nrow(data)
train_index <- sample(
  1:n,
  size=0.8*n
)
train <- data[train_index,]test <- data[-train_index,]
# Logistic regressionmodel <- glm(
  Species ~ Petal.Length + Petal.Width,
  data=train,
  family=binomial
)
```

```

# Model summary
print(summary(model))
# Predict probabilities
prob <- predict(
  model,
  newdata=test,
  type="response"
)
# Convert probability into class
pred <- ifelse(
  prob > 0.5,
  "setosa",
  "non-setosa"
)
pred <- factor(
  pred,
  levels=levels(test$Species)
)
# Confusion matrix
confusion_matrix <- table(
  Actual=test$Species,
  Predicted=pred
)
print(confusion_matrix)
# Calculate accuracy
accuracy <- sum(diag(confusion_matrix)) /
  sum(confusion_matrix)
print(accuracy)

```

## Result

Thus, the Iris dataset was successfully divided into training and testing sets, a logistic regression model was built, predictions were made and model performance was evaluated using a confusion matrix and accuracy.